

```

=====
=====
Anomie's SPC700 Doc
$Revision: 1162 $
$Date: 2009-07-19 21:25:30 -0400 (Sun, 19 Jul 2009) $
<anomie@users.sourceforge.net>
=====
=====

```

IPL BOOT ROM

```
=====
```

This is the boot ROM image, which the SPC700 executes on reset.

```

$78      $CD $EF $BD $E8 $00 $C6 $1D $D0 $FC $8F $AA $F4 $8F $BB $F5
$F5      $CC $F4 $D0 $FB $2F $19 $EB $F4 $D0 $FC $7E $F4 $D0 $0B $E4
$BA      $CB $F4 $D7 $00 $FC $D0 $F3 $AB $01 $10 $EF $7E $F4 $10 $EB
$FF      $F6 $DA $00 $BA $F4 $C4 $F4 $DD $5D $D0 $DB $1F $00 $00 $C0

```

```

.ORG $FFC0
      MOV X, #$EF      ; *** INIT ***
      MOV SP, X        ; setup stack
      MOV A, #$00      ; clear page 0 RAM
-      MOV (X),A
      DEC X
      BNE -
      MOV $F4,$$AA     ; Signal "ready" to 5A22: $2140-1 will return
#$BBAA
      MOV $F5,$$BB
-      CMP $F4,$$CC     ; wait for 5A22 to write $$CC to $2140
      BNE -
      BRA Start
Trans: MOV Y,$F4        ; *** TRANSFER ROUTINE ***
      BNE Trans        ; First, wait for 5A22 to indicate Byte 0
ready on $2140
-      CMP Y,$F4        ; start loop: wait for "next byte/end" signal
on $2140
      BNE +
      MOV A,$F5         ; Got "next byte" ($2140 matches expected
byte index)
      MOV $F4,Y         ; Read byte-to-write from $2141, echo $2140
to signal
      MOV [$00]+Y,A     ; ready, and write the byte and update the
counter.
      INC Y
      BNE -
      INC $01          ; (handle $xxFF->$xx00 overflow case on
increment)
+      BPL -
      CMP Y,$F4        ; If "next byte/end" is not equal to expected

```

```

next byte
    BPL -          ; index, it's "end": drop back into the main
loop.
Start:  MOVW YA,$F6    ; *** MAIN LOOP ***
        MOVW $00,YA   ; Get address from 5A22's $2142-3,
        MOVW YA,$F4   ; mode from $2141, and echo $2140 back
        MOV $F4,A
        MOV A,Y
        MOV X,A
        BNE Trans     ; Mode non-0: begin transfer
        JMP [$0000+X]  ; Mode 0: jump to address
        .DW $FFC0      ; RESET vector

```

To properly manipulate this into uploading your data, the following procedure seems to work:

1. Wait for a 16-bit read on \$2140-1 to return \$BBAA.
2. Write the target address to \$2142-3.
3. Write non-zero to \$2141.
4. Write \$CC to \$2140.
5. Wait until reading \$2140 returns \$CC.
6. Set your first byte to \$2141.
7. Set your byte index (\$00 for the first byte) to \$2140.
8. Wait for \$2140 to echo your byte index.
9. Go back to step 6 with your next byte and ++index until you're done.
10. If you want to write another block, write the next address to \$2142-3, non-zero to \$2141, and index+2 (or +3 if that would be zero, otherwise it'll screw up the next transfer) to \$2140 and wait for the echo. Then go to step 6 with index=0.
11. Otherwise, you can jump to some code you've just uploaded. Put the target address in \$2142-3, \$00 in \$2141, and index+2 in \$2140 and wait for the echo. Shortly afterwards, your code will be executing.

After power on, on entry to the first user code loaded the registers have the following values: A=0, X=0, Y=0, PSW=\$02, SP=\$EF

If you ever want to go back to this uploader, simply jump to \$FFC0 with IPL enabled and the P flag clear (or \$FFC9 to skip resetting the stack and page 0).

SPC700 REGISTERS

=====

The SPC700 registers are memory-mapped to the range \$00f0 to \$00ff. Write-only

registers always read back as 0. These registers act as an overlay on the underlying RAM much as the IPL ROM does: writes affect both the register and the underlying RAM, while reads read the register value.

\$00f0 -w TEST - Testing functions
ssssTRrt

ssss = CPU speed control (doesn't affect timer rate)

0 = normal CPU rate
1 = 3/5 normal rate
2 = 3/9 normal rate
3 = 3/17 normal rate
4 = 3/4 normal rate
5 = 3/6 normal rate
6 = 3/10 normal rate
7 = 3/18 normal rate
8 = 3/6 normal rate
9 = 3/8 normal rate
A = 3/12 normal rate
B = 3/20 normal rate
C = 3/10 normal rate
D = 3/12 normal rate
E = 3/16 normal rate
F = 3/24 normal rate

** Settings other than 0 may lock up the SPC700! **

r = Clearing it disables writes to RAM. Only \$f0-\$ff may be effectively written. This also disables S-DSP buffer writes.

R = Unknown, but setting it basically locks the SPC700.
Perhaps this disables reads from RAM, so the SPC700 ends up executing some random garbage instruction over and over?

T/t = Unknown, but timers do not work unless T=1 and t=0.

This register's behavior hasn't been thoroughly tested.

Playing with this register can have all sorts of unusual effects, including changing the "sync" between when the S-DSP generates samples and when the timers tick. I would not be surprised at all if these effects so vary depending on the versions of the SPC700 and the S-DSP.

The best advice I can give is to not touch this register in your SPC700 code, and print lots of warnings from your S-APU emulator if

any value
other than \$0A is written.

On power on this register contains #\$0A. Also, writing this
register seems to have no effect when the P flag is set.

\$00f1 -w CONTROL - I/O and Timer Control
r-ba-210

r = When set, the 64-byte IPL ROM can be read from \$FFC0-\$FFFF. When
clear, this area is normal RAM. Note that writes to
\$FFC0-\$FFFF affect the RAM regardless of this setting.

a = When 1 is written, input ports \$00f4 and \$00f5 are
cleared to \$00.

b = When 1 is written, input ports \$00f6 and \$00f7 are
cleared to \$00.

Note that this is a one-time zeroing, and does not
affect the values read by S-CPU. Also, note that the zeroing occurs
whenever 1 is written, not on a 0->1 transition.

012 = Enable timer 0, 1, or 2. See registers \$00fa-f. When
transitioning from 0 to 1, the timer Stage 2 and 3
counters are both reset to 0. Note however that the Stage 1 'counter'
is not reset.

On power on or reset, it seems to be set to #\$B0.

\$00f2 rw DSPADDR - DSP Communication Address
aaaaaaaa

\$00f3 rw DSPDATA - DSP Communication Data
dddddddd

These registers control access to the DSP. When a value is
written to \$00f2, the value of the corresponding DSP register may be
read from \$00f3, or a new value may be written to \$00f3. Writes beyond
address \$7f are ignored, while reads mask the address with \$7f.

\$00f4 r- CPUI0 - CPU Input Register 0

\$00f4 -w CPUO0 - CPU Output Register 0

\$00f5 r- CPUI1 - CPU Input Register 1

```

$00f5 -w CPU01 - CPU Output Register 1
$00f6 r- CPUI2 - CPU Input Register 2
$00f6 -w CPU02 - CPU Output Register 2
$00f7 r- CPUI3 - CPU Input Register 3
$00f7 -w CPU03 - CPU Output Register 3
xxxxxxx

```

These registers are used in communication with the 5A22 S-CPU. There are eight total registers accessed by these four addresses: four write-only output ports to the S-CPU and four read-only input ports from the S-CPU. Writing a value to an output port doesn't affect the value in the corresponding input port; the SPC700 can modify the input ports only by clearing them using the CONTROL register.

If the SPC700 writes to an output port while the S-CPU is reading it, the S-CPU will read the logical OR of the old and new values. The exact cycles during which the 'read' actually occurs is not known, although a good guess would be some portion of the final 3 master cycles of the 6-cycle S-CPU memory access. Possibly the same thing happens the other way around, but the details are unknown.

```

$00f8 rw - Normal RAM
$00f9 rw - Normal RAM

```

These registers act like RAM, except that they can still be written when \$F0 bit 1 is set and are not altered by S-DSP echo buffer writes.

```

$00fa -w T0TARGET - Timer 0 Scaling Target
$00fb -w T1TARGET - Timer 1 Scaling Target
$00fc -w T2TARGET - Timer 2 Scaling Target
      tttttttt
$00fd r- T0OUT - Timer 0 Output
$00fe r- T1OUT - Timer 1 Output
$00ff r- T2OUT - Timer 2 Output
      0000xxxx

```

The SPC700 has 3 timers, two (#0 and #1) with a base rate of 128 clock cycles (~8000Hz) and one (#2) with a base rate of 16 clock cycles

(~64000Hz).

Each timer consists of 3 stages: (thanks TRAC)

Stage 1: 128:1 (T0, T1) or 16:1 (T2) scaler.

Stage 2: 1-256 'divisor', based on a 0-255 wraparound counter and a

post-increment comparator.

Stage 3: The 4-bit counter for output ticks from the comparater

stage.

Stage 1 runs constantly, and cannot be stopped or reset.

Stage 2

increments each 'tick' of Stage 1 when the timer is enabled; if the new

value is equal to the value in TnTARGET, a 'tick' is passed on to Stage

3 and Stage 2 is zeroed. Stage 3 may be read from TnOUT, and the value

is zeroed on read.

Stage 1 ticks for T0 and T1 occur at the same time and at the same time

as a T2 tick. Unless the TEST register has been played with, this

occurs 2 cycles after the S-DSP writes the right channel into the echo

buffer. T2, of course, will also have a Stage 1 tick 18 cycles after

that echo buffer write.

Note that a target value of \$00 corresponds to 256 ticks, and that the

output value is limited to 4 bits.

Changing the target values while the timer is running CAN be done.

However, note that if the Stage 2 counter is 2 and you set T=1, a

Stage 3 'tick' will not occur until the Stage 2 counter wraps all the

way back around to 1...

Reading the TnTARGET registers always returns 0. Writes to TnOUT

registers have no effect (but note that most "write" opcodes also

read).

On power on, all three TnOUT have the value \$F. On reset, they are \$0.

On power on, all three TnTARGET have the value \$0. On reset, they retain their old values.

There is a race condition when writing the TnTARGET registers just after the Stage 1 tick when the written value matches the new Stage 2 counter value. See the email blargg-TnTARGET-glitch-email.txt for details.

OPCODES =====

In the below,

(N) means the byte (or word when used with YA) at N.

[N] means the word address at N.

d is a direct-page address.

a is an absolute address.

m.b indicates that the 16-bit operand specifies a 13-bit absolute address with

the high 3 bits indicating which bit of the addressed byte is to be

affected.

d.# indicates that only bit # of the byte at direct page address d is to be

affected.

Operands are encoded little endian, with multiple operands stored last to

first. For example, "OP A, B" is stored in memory as "OP B A".

Mnemonics are

represented as "OP dest, src" where applicable. However, there are a few

exceptions:

* BBC, BBS, CBNE, and DBNZ all store the 'r' as the second byte.

For example,

BBC \$01.0, \$02 would be stored as "13 01 02".

DAA and DAS depend on C and H: First, if $A > 0x99$ or Carry/Borrow, add/sub 0x60

and set Carry/Borrow. Then if $(A \& 0x0f) > 9$ or HalfCarry/HalfBorrow, add/sub 0x06

(but don't change HalfCarry/Borrow).

DIV has some interesting corner cases. ZN are set based on A. V is set if

$YA/X > \$FF$ (so the result won't fit in A). H is odd, it seems to get set based on

$X \& \$F \leq Y \& \F . The result is correct as long as $YA/X < 0x200$, otherwise Y and A are

not helpful. An algorithm:

uint17 yva=YA

uint17 x=X<<9

loop 9 times {

```

    ROL yva
    if(yva>=x) yva=yva XOR 1
    if(yva&1) yva-=x          // remember, clip to 17 bits
}
yva => Y, A, and V flag as YYYYYYYY V AAAAAAAA

```

Execution and instructions will wrap from \$ffff to \$0000. Direct page accesses will wrap within the direct page (page \$00 or \$01, depending on the P flag). The stack is always in page 1, and will wrap as such always.

Most of the MOV instructions targeting memory actually include a read cycle on the destination address in addition to the expected write cycle. For example, "MOV \$ff, #\$00" will read from \$ff at some point, and therefore will reset T2OUT. OTOH, "MOV \$ff, \$00" won't. MOVW does a read on the low byte of the destination only. Note that none of this applies to SET1, CLR1, OR, or any other opcode, since those are all RMW instructions.

Mnemonic	Code	Bytes	Cyc	Operation
NVPBHZC				
ADC (X), (Y)	99	1	5	(X) = (X)+(Y)+C
NV..H.ZC				
ADC A, #i	88	2	2	A = A+i+C
NV..H.ZC				
ADC A, (X)	86	1	3	A = A+(X)+C
NV..H.ZC				
ADC A, [d]+Y	97	2	6	A = A+([d]+Y)+C
NV..H.ZC				
ADC A, [d+X]	87	2	6	A = A+([d+X])+C
NV..H.ZC				
ADC A, d	84	2	3	A = A+(d)+C
NV..H.ZC				
ADC A, d+X	94	2	4	A = A+(d+X)+C
NV..H.ZC				
ADC A, !a	85	3	4	A = A+(a)+C
NV..H.ZC				
ADC A, !a+X	95	3	5	A = A+(a+X)+C
NV..H.ZC				
ADC A, !a+Y	96	3	5	A = A+(a+Y)+C
NV..H.ZC				
ADC dd, ds	89	3	6	(dd) = (dd)+(d)+C
NV..H.ZC				
ADC d, #i	98	3	5	(d) = (d)+i+C
NV..H.ZC				

ADDW YA, d	7A	2	5	YA = YA + (d), H on high byte
NV...H.ZC				

AND (X), (Y)	39	1	5	(X) = (X) & (Y)
N.....Z.				

AND A, #i	28	2	2	A = A & i
N.....Z.				

AND A, (X)	26	1	3	A = A & (X)
N.....Z.				

AND A, [d]+Y	37	2	6	A = A & ([d]+Y)
N.....Z.				

AND A, [d+X]	27	2	6	A = A & ([d+X])
N.....Z.				

AND A, d	24	2	3	A = A & (d)
N.....Z.				

AND A, d+X	34	2	4	A = A & (d+X)
N.....Z.				

AND A, !a	25	3	4	A = A & (a)
N.....Z.				

AND A, !a+X	35	3	5	A = A & (a+X)
N.....Z.				

AND A, !a+Y	36	3	5	A = A & (a+Y)
N.....Z.				

AND dd, ds	29	3	6	(dd) = (dd) & (ds)
N.....Z.				

AND d, #i	38	3	5	(d) = (d) & i
N.....Z.				

AND1 C, /m.b	6A	3	4	C = C &
~(m.b)		C		

AND1 C, m.b	4A	3	4	C = C &
(m.b)		C		

ASL A	1C	1	2	Left shift A: high->C, 0->low
N.....ZC				

ASL d	0B	2	4	Left shift (d) as above
N.....ZC				

ASL d+X	1B	2	5	Left shift (d+X) as above
N.....ZC				

ASL !a	0C	3	5	Left shift (a) as above
N.....ZC				

BBC d.0, r	13	3	5/7	PC+=r if d.0 ==
0				

BBC d.1, r	33	3	5/7	PC+=r if d.1 ==
0				

BBC d.2, r	53	3	5/7	PC+=r if d.2 ==
0				

BBC d.3, r	73	3	5/7	PC+=r if d.3 ==
0				

BBC d.4, r	93	3	5/7	PC+=r if d.4 ==
0				

BBC d.5, r	B3	3	5/7	PC+=r if d.5 ==
0				

0	BBC	d.6, r	 D3	3	5/7	PC+=r	if d.6 ==
0	BBC	d.7, r	 F3	3	5/7	PC+=r	if d.7 ==
0							
	BBS	d.0, r	03	3	5/7	PC+=r	if d.0 ==
1	BBS	d.1, r	 23	3	5/7	PC+=r	if d.1 ==
1	BBS	d.2, r	 43	3	5/7	PC+=r	if d.2 ==
1	BBS	d.3, r	 63	3	5/7	PC+=r	if d.3 ==
1	BBS	d.4, r	 83	3	5/7	PC+=r	if d.4 ==
1	BBS	d.5, r	 A3	3	5/7	PC+=r	if d.5 ==
1	BBS	d.6, r	 C3	3	5/7	PC+=r	if d.6 ==
1	BBS	d.7, r	 E3	3	5/7	PC+=r	if d.7 ==
1							
	BCC	r	90	2	2/4	PC+=r	if C ==
0							
	BCS	r	B0	2	2/4	PC+=r	if C ==
1							
	BEQ	r	F0	2	2/4	PC+=r	if Z ==
1							
	BMI	r	30	2	2/4	PC+=r	if N ==
1							
	BNE	r	D0	2	2/4	PC+=r	if Z ==
0							
	BPL	r	10	2	2/4	PC+=r	if N ==
0							
	BVC	r	50	2	2/4	PC+=r	if V ==
0							
	BVS	r	70	2	2/4	PC+=r	if V ==
1							
	BRA	r	2F	2	4		
	PC+=r						
	BRK		0F	1	8	Push PC and Flags, PC =	
	[\$FFDE]	...1.0..					
	CALL	!a	3F	3	8	(SP--)=PCh, (SP--)=PCl,	
	PC=a						
	CBNE	d+X, r	DE	3	6/8	CMP A, (d+X) then	
	BNE						
	CBNE	d, r	2E	3	5/7	CMP A, (d) then	
	BNE						
	CLR1	d.0	12	2	4	d.0 =	

0	CLR1 d.1	32	2	4	d.1 =
0	CLR1 d.2	52	2	4	d.2 =
0	CLR1 d.3	72	2	4	d.3 =
0	CLR1 d.4	92	2	4	d.4 =
0	CLR1 d.5	B2	2	4	d.5 =
0	CLR1 d.6	D2	2	4	d.6 =
0	CLR1 d.7	F2	2	4	d.7 =
0					
	CLRC	60	1	2	C =
0	CLRP	20	1	2	P =
0	CLR V	E0	1	2	V = 0, H =
0					
	CMP (X), (Y)	79	1	5	(X) - (Y)
N.....ZC					
	CMP A, #i	68	2	2	A - i
N.....ZC					
	CMP A, (X)	66	1	3	A - (X)
N.....ZC					
	CMP A, [d]+Y	77	2	6	A - ([d]+Y)
N.....ZC					
	CMP A, [d+X]	67	2	6	A - ([d+X])
N.....ZC					
	CMP A, d	64	2	3	A - (d)
N.....ZC					
	CMP A, d+X	74	2	4	A - (d+X)
N.....ZC					
	CMP A, !a	65	3	4	A - (a)
N.....ZC					
	CMP A, !a+X	75	3	5	A - (a+X)
N.....ZC					
	CMP A, !a+Y	76	3	5	A - (a+Y)
N.....ZC					
	CMP X, #i	C8	2	2	X - i
N.....ZC					
	CMP X, d	3E	2	3	X - (d)
N.....ZC					
	CMP X, !a	1E	3	4	X - (a)
N.....ZC					
	CMP Y, #i	AD	2	2	Y - i
N.....ZC					
	CMP Y, d	7E	2	3	Y - (d)
N.....ZC					
	CMP Y, !a	5E	3	4	Y - (a)

N.....ZC				
CMP dd, ds	69	3	6	(dd) - (ds)
N.....ZC				
CMP d, #i	78	3	5	(d) - i
N.....ZC				
CMPW YA, d	5A	2	4	YA - (d)
N.....ZC				
DAA A	DF	1	3	decimal adjust for addition
N.....ZC				
DAS A	BE	1	3	decimal adjust for subtraction
N.....ZC				
DBNZ Y, r	FE	2	4/6	Y-- then
JNZ				
DBNZ d, r	6E	3	5/7	(d)-- then
JNZ				
DEC A	9C	1	2	A--
N.....Z.				
DEC X	1D	1	2	X--
N.....Z.				
DEC Y	DC	1	2	Y--
N.....Z.				
DEC d	8B	2	4	(d)--
N.....Z.				
DEC d+X	9B	2	5	(d+X)--
N.....Z.				
DEC !a	8C	3	5	(a)--
N.....Z.				
DECW d	1A	2	6	Word (d)--
N.....Z.				
DI	C0	1	3	I =
0			0..	
DIV YA, X	9E	1	12	A=YA/X, Y=mod(YA,X)
NV..H.Z.				
EI	A0	1	3	I =
1			1..	
EOR (X), (Y)	59	1	5	(X) = (X) EOR (Y)
N.....Z.				
EOR A, #i	48	2	2	A = A EOR i
N.....Z.				
EOR A, (X)	46	1	3	A = A EOR (X)
N.....Z.				
EOR A, [d]+Y	57	2	6	A = A EOR ([d]+Y)
N.....Z.				
EOR A, [d+X]	47	2	6	A = A EOR ([d+X])
N.....Z.				

EOR A, d	44	2	3	A = A EOR (d)
N.....Z.				
EOR A, d+X	54	2	4	A = A EOR (d+X)
N.....Z.				
EOR A, !a	45	3	4	A = A EOR (a)
N.....Z.				
EOR A, !a+X	55	3	5	A = A EOR (a+X)
N.....Z.				
EOR A, !a+Y	56	3	5	A = A EOR (a+Y)
N.....Z.				
EOR dd, ds	49	3	6	(dd) = (dd) EOR (ds)
N.....Z.				
EOR d, #i	58	3	5	(d) = (d) EOR i
N.....Z.				
EOR1 C, m.b	8A	3	5	C = C EOR
(m.b)	C			
INC A	BC	1	2	A++
N.....Z.				
INC X	3D	1	2	X++
N.....Z.				
INC Y	FC	1	2	Y++
N.....Z.				
INC d	AB	2	4	(d)++
N.....Z.				
INC d+X	BB	2	5	(d+X)++
N.....Z.				
INC !a	AC	3	5	(a)++
N.....Z.				
INCW d	3A	2	6	Word (d)++
N.....Z.				
JMP [!a+X]	1F	3	6	PC =
[a+X]				
JMP !a	5F	3	3	PC =
a				
LSR A	5C	1	2	Right shift A: 0->high, low->C
N.....ZC				
LSR d	4B	2	4	Right shift (d) as above
N.....ZC				
LSR d+X	5B	2	5	Right shift (d+X) as above
N.....ZC				
LSR !a	4C	3	5	Right shift (a) as above
N.....ZC				
MOV (X)+, A	AF	1	4	(X++) = A (no
read)				
MOV (X), A	C6	1	4	(X) = A
(read)				
MOV [d]+Y, A	D7	2	7	([d]+Y) = A
(read)				

MOV	[d+X], A	C7	2	7	([d+X]) = A	
(read)						
MOV	A, #i	E8	2	2	A = i	
N.....Z.						
MOV	A, (X)	E6	1	3	A = (X)	
N.....Z.						
MOV	A, (X)+	BF	1	4	A = (X++)	
N.....Z.						
MOV	A, [d]+Y	F7	2	6	A = ([d]+Y)	
N.....Z.						
MOV	A, [d+X]	E7	2	6	A = ([d+X])	
N.....Z.						
MOV	A, X	7D	1	2	A = X	
N.....Z.						
MOV	A, Y	DD	1	2	A = Y	
N.....Z.						
MOV	A, d	E4	2	3	A = (d)	
N.....Z.						
MOV	A, d+X	F4	2	4	A = (d+X)	
N.....Z.						
MOV	A, !a	E5	3	4	A = (a)	
N.....Z.						
MOV	A, !a+X	F5	3	5	A = (a+X)	
N.....Z.						
MOV	A, !a+Y	F6	3	5	A = (a+Y)	
N.....Z.						
MOV	SP, X	BD	1	2	SP =	
X						
MOV	X, #i	CD	2	2	X = i	
N.....Z.						
MOV	X, A	5D	1	2	X = A	
N.....Z.						
MOV	X, SP	9D	1	2	X = SP	
N.....Z.						
MOV	X, d	F8	2	3	X = (d)	
N.....Z.						
MOV	X, d+Y	F9	2	4	X = (d+Y)	
N.....Z.						
MOV	X, !a	E9	3	4	X = (a)	
N.....Z.						
MOV	Y, #i	8D	2	2	Y = i	
N.....Z.						
MOV	Y, A	FD	1	2	Y = A	
N.....Z.						
MOV	Y, d	EB	2	3	Y = (d)	
N.....Z.						
MOV	Y, d+X	FB	2	4	Y = (d+X)	
N.....Z.						
MOV	Y, !a	EC	3	4	Y = (a)	
N.....Z.						
MOV	dd, ds	FA	3	5	(dd) = (ds)	(no
read)						
MOV	d+X, A	D4	2	5	(d+X) = A	
(read)						

MOV	d+X, Y	DB	2	5	(d+X) = Y
(read)					
MOV	d+Y, X	D9	2	5	(d+Y) = X
(read)					
MOV	d, #i	8F	3	5	(d) = i
(read)					
MOV	d, A	C4	2	4	(d) = A
(read)					
MOV	d, X	D8	2	4	(d) = X
(read)					
MOV	d, Y	CB	2	4	(d) = Y
(read)					
MOV	!a+X, A	D5	3	6	(a+X) = A
(read)					
MOV	!a+Y, A	D6	3	6	(a+Y) = A
(read)					
MOV	!a, A	C5	3	5	(a) = A
(read)					
MOV	!a, X	C9	3	5	(a) = X
(read)					
MOV	!a, Y	CC	3	5	(a) = Y
(read)					
MOV1	C, m.b	AA	3	4	C =
(m.b)				C	
MOV1	m.b, C	CA	3	6	(m.b) =
C					
MOVW	YA, d	BA	2	5	YA = word (d)
N.....Z.					
MOVW	d, YA	DA	2	5	word (d) = YA (read low
only)					
MUL	YA	CF	1	9	YA = Y * A, NZ on Y only
N.....Z.					
NOP		00	1	2	do
nothing					
NOT1	m.b	EA	3	5	m.b =
~m.b					
NOTC		ED	1	3	C = !
C				C	
OR	(X), (Y)	19	1	5	(X) = (X) (Y)
N.....Z.					
OR	A, #i	08	2	2	A = A i
N.....Z.					
OR	A, (X)	06	1	3	A = A (X)
N.....Z.					
OR	A, [d]+Y	17	2	6	A = A ([d]+Y)
N.....Z.					
OR	A, [d+X]	07	2	6	A = A ([d+X])

N.....Z.	OR A, d	04	2	3	A = A (d)
N.....Z.	OR A, d+X	14	2	4	A = A (d+X)
N.....Z.	OR A, !a	05	3	4	A = A (a)
N.....Z.	OR A, !a+X	15	3	5	A = A (a+X)
N.....Z.	OR A, !a+Y	16	3	5	A = A (a+Y)
N.....Z.	OR dd, ds	09	3	6	(dd) = (dd) (ds)
N.....Z.	OR d, #i	18	3	5	(d) = (d) i
N.....Z.					
	OR1 C, /m.b	2A	3	5	C = C
~(m.b)				C	
	OR1 C, m.b	0A	3	5	C = C
(m.b)				C	
	PCALL u	4F	2	6	CALL
\$FF00+u					
	POP A	AE	1	4	A = (+
+SP)					
	POP PSW	8E	1	4	Flags = (++SP)
NVPBHZC					
	POP X	CE	1	4	X = (+
+SP)					
	POP Y	EE	1	4	Y = (+
+SP)					
	PUSH A	2D	1	4	(SP--) =
A					
	PUSH PSW	0D	1	4	(SP--) =
Flags					
	PUSH X	4D	1	4	(SP--) =
X					
	PUSH Y	6D	1	4	(SP--) =
Y					
	RET	6F	1	5	Pop
PC					
	RET1	7F	1	6	Pop Flags, PC
NVPBHZC					
	ROL A	3C	1	2	Left shift A: low=C, C=high
N.....ZC					
	ROL d	2B	2	4	Left shift (d) as above
N.....ZC					
	ROL d+X	3B	2	5	Left shift (d+X) as above
N.....ZC					
	ROL !a	2C	3	5	Left shift (a) as above

N.....ZC

ROR A	7C	1	2	Right shift A: high=C, C=low
N.....ZC				
ROR d	6B	2	4	Right shift (d) as above
N.....ZC				
ROR d+X	7B	2	5	Right shift (d+X) as above
N.....ZC				
ROR !a	6C	3	5	Right shift (a) as above
N.....ZC				

SBC (X), (Y)	B9	1	5	(X) = (X)-(Y)-!C
NV..H.ZC				
SBC A, #i	A8	2	2	A = A-i-!C
NV..H.ZC				
SBC A, (X)	A6	1	3	A = A-(X)-!C
NV..H.ZC				
SBC A, [d]+Y	B7	2	6	A = A-([d]+Y)-!C
NV..H.ZC				
SBC A, [d+X]	A7	2	6	A = A-([d+X])-!C
NV..H.ZC				
SBC A, d	A4	2	3	A = A-(d)-!C
NV..H.ZC				
SBC A, d+X	B4	2	4	A = A-(d+X)-!C
NV..H.ZC				
SBC A, !a	A5	3	4	A = A-(a)-!C
NV..H.ZC				
SBC A, !a+X	B5	3	5	A = A-(a+X)-!C
NV..H.ZC				
SBC A, !a+Y	B6	3	5	A = A-(a+Y)-!C
NV..H.ZC				
SBC dd, ds	A9	3	6	(dd) = (dd)-(ds)-!C
NV..H.ZC				
SBC d, #i	B8	3	5	(d) = (d)-i-!C
NV..H.ZC				

SET1 d.0	02	2	4	d.0 =
1				
SET1 d.1	22	2	4	d.1 =
1				
SET1 d.2	42	2	4	d.2 =
1				
SET1 d.3	62	2	4	d.3 =
1				
SET1 d.4	82	2	4	d.4 =
1				
SET1 d.5	A2	2	4	d.5 =
1				
SET1 d.6	C2	2	4	d.6 =
1				
SET1 d.7	E2	2	4	d.7 =
1				
SETC	80	1	2	C =

1 1 SETP	40	1	2	1 P =
				..1.....
SLEEP	EF	1	?	Halts the
processor				
STOP	FF	1	?	Halts the
processor				
SUBW YA, d	9A	2	5	YA = YA - (d), H on high byte
NV..H.ZC				
TCALL 0	01	1	8	CALL
[\$FFDE]				
TCALL 1	11	1	8	CALL
[\$FFDC]				
TCALL 2	21	1	8	CALL
[\$FFDA]				
TCALL 3	31	1	8	CALL
[\$FFD8]				
TCALL 4	41	1	8	CALL
[\$FFD6]				
TCALL 5	51	1	8	CALL
[\$FFD4]				
TCALL 6	61	1	8	CALL
[\$FFD2]				
TCALL 7	71	1	8	CALL
[\$FFD0]				
TCALL 8	81	1	8	CALL
[\$FFCE]				
TCALL 9	91	1	8	CALL
[\$FFCC]				
TCALL 10	A1	1	8	CALL
[\$FFCA]				
TCALL 11	B1	1	8	CALL
[\$FFC8]				
TCALL 12	C1	1	8	CALL
[\$FFC6]				
TCALL 13	D1	1	8	CALL
[\$FFC4]				
TCALL 14	E1	1	8	CALL
[\$FFC2]				
TCALL 15	F1	1	8	CALL
[\$FFC0]				
TCLR1 !a	4E	3	6	(a) = (a)&~A, ZN as for A-(a)
N.....Z.				
TSET1 !a	0E	3	6	(a) = (a) A, ZN as for A-(a)
N.....Z.				
XCN A	9F	1	5	A = (A>>4) (A<<4)
N.....Z.				